

Windows应用程序与MFC基础

党宇

dangyu@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南开大学

Nankai University

主要内容

- 初识可视化编程
- Windows应用程序
 - 对应课本：第三章
 - 补充内容：《深入浅出MFC》第一章
- MFC基础知识
 - 对于课本：第八章
 - 补充内容：《VC++深入详解》第三、四章

主要内容

- 初识可视化编程
- Windows应用程序
 1. Windows程序开发流程
 2. 事件与消息
 3. Windows应用程序的基本结构
 4. 应用程序举例
- MFC基础知识

主要内容

- 初识可视化编程
- Windows应用程序
- MFC基础知识
 1. MFC类的组织结构
 2. MFC框架程序分析
 3. MFC消息映射机制

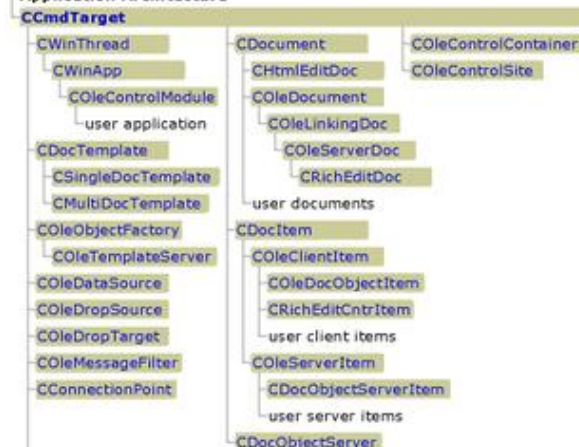
1、MFC类的组织结构

- 当前版本包含100多个类，可实现应用程序大部分功能
- MFC是一个应用程序框架，帮助定义应用程序的结构，简化Windows编程难度
- MFC类以层次结构组织，封装大部分Windows API和控件
- **MFC类的组织结构 (书P.164)**
 - 1) 根类：CObject
 - 2) 应用程序体系结构类
 - 3) 可视对象类
 - 4) 通用类
 - 5) OLE类
 - 6) ODBC数据库类

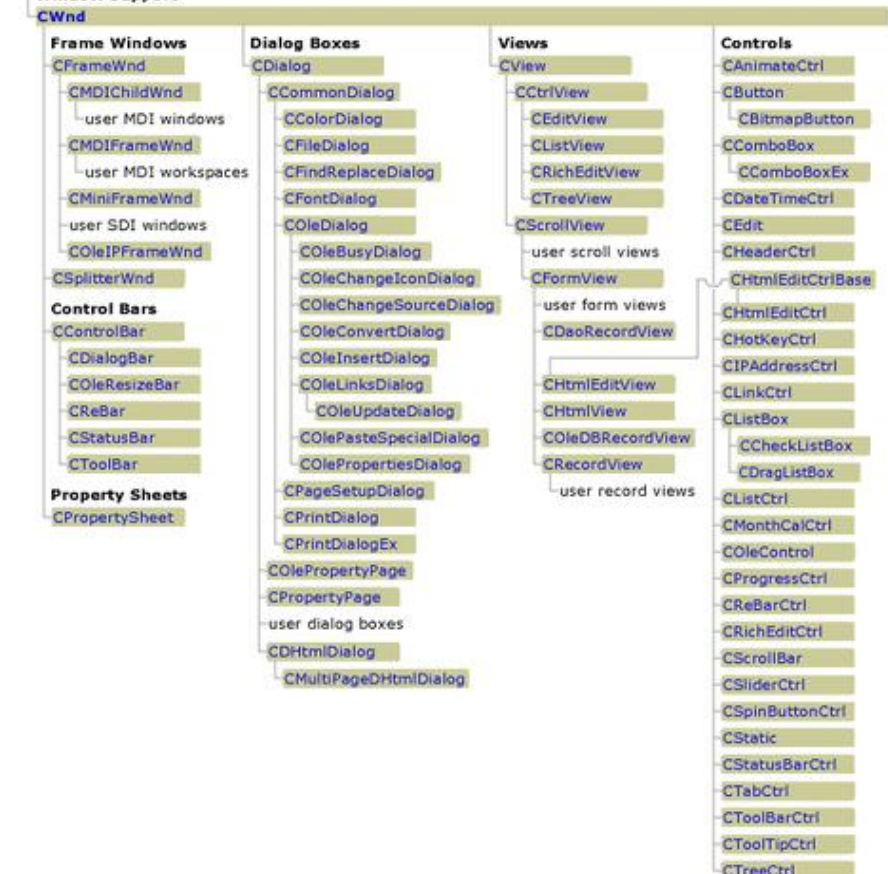
Microsoft Foundation Class Library 6.0

Object

Application Architecture



Window Support



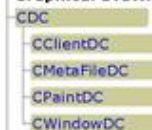
Exceptions



File Services



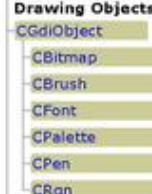
Graphical Drawing



Control Support



Graphical Drawing Objects



Menus



Command Line



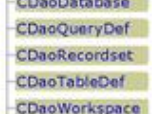
ODBC Database Support



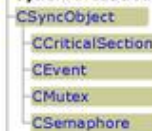
Long Binary



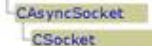
DAO Database Support



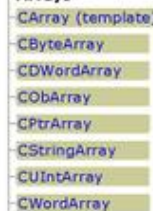
Synchronization



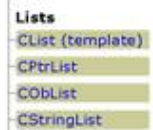
Windows Sockets



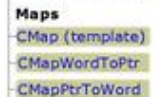
Arrays



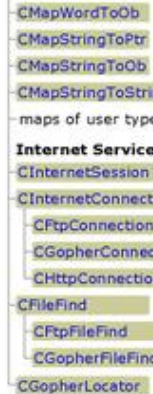
Lists



Maps



Internet Services

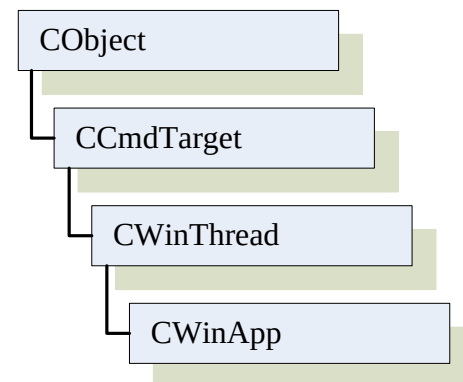


Classes Not Derived from CObject



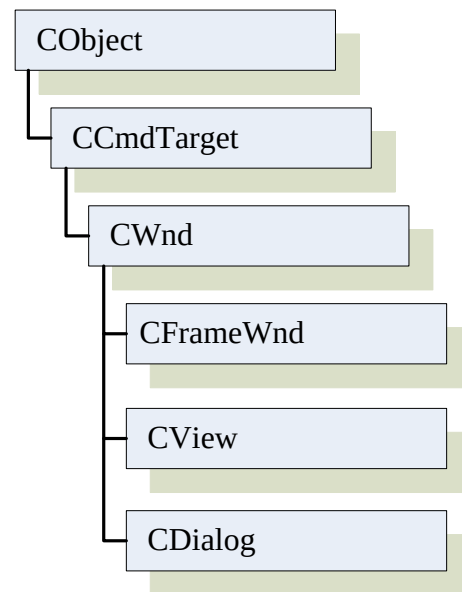
1、MFC类的组织结构

- CCmdTarget: MFC所有具有消息映射属性的基类
- CWinThread: MFC线程基类
- CWinApp: MFC应用程序基类 (**重要**)
 - 封装初始化、运行和终止应用程序的代码
 - 成员函数InitInstance: 执行程序初始化操作
 - 成员函数Run: 启动缺省的消息循环
 - 每个MFC程序有且仅有一个从CWinApp派生的类, 每一个MFC程序实例有且仅有一个该派生类的实例化对象, 用于表示应用程序本身

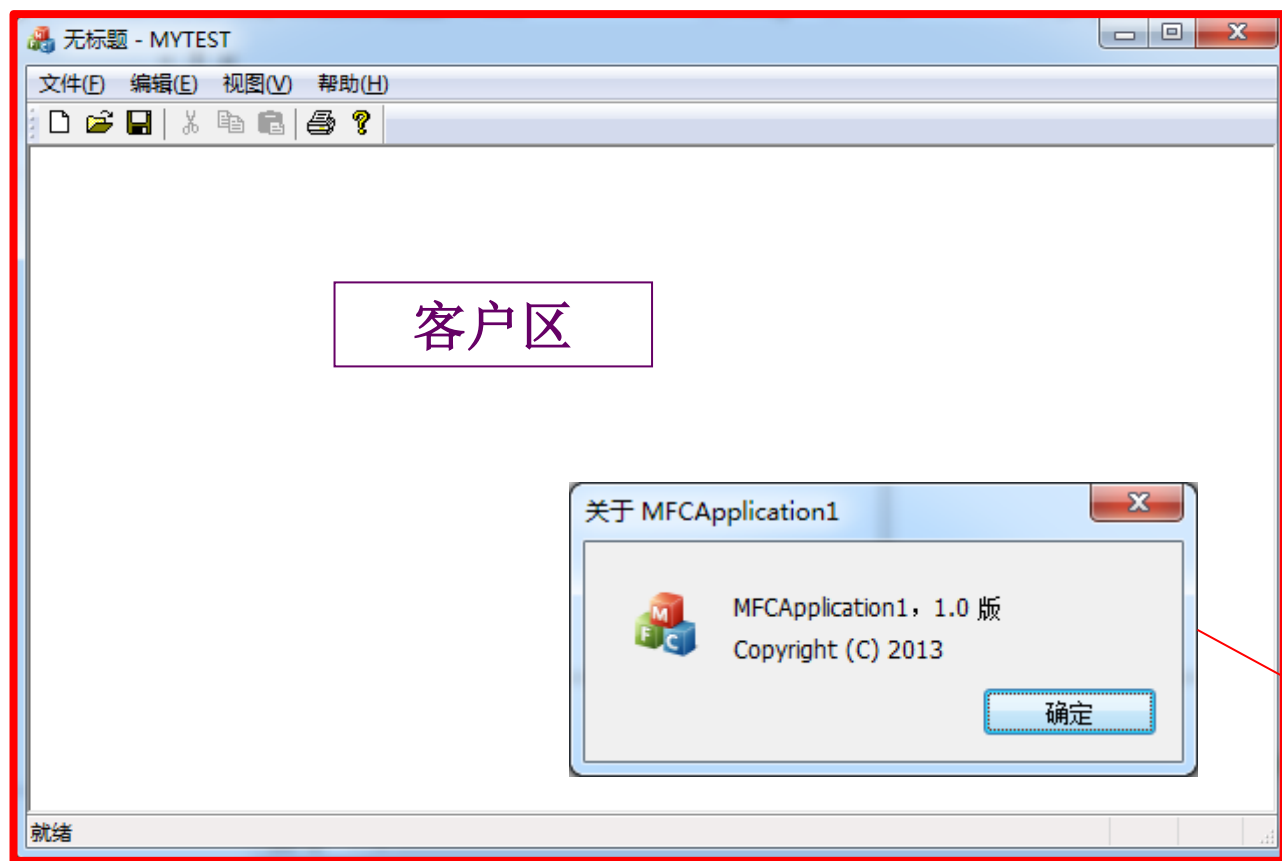


1、MFC类的组织结构

- **CWnd**: MFC窗口基类，封装与窗口相关的操作
创建Windows窗口的步骤
 - 构造一个CWnd类对象
 - 调用Create建立Windows窗口，并将它连到CWnd对象上
- **CFrameWnd**: MFC框架窗口类，是单文档框架窗口的基类
- **CView**: MFC视图基类，实现框架窗口中的客户区
- **CDialog**: MFC通用对话框类



窗口



客户区

框架窗口

对话框

1、MFC类的组织结构

- **CDocument**: 显示文档数据的应用程序专用文档基类
- **CDocTemplate**: 文档模板基类，协调文档、视图和框架窗口的创建
 - 文档对象由文档模板对象创建，管理应用程序的数据，文档操作包括文档创建、打开与保存
 - 文档模板指定了文档所需资源，每个文档对象都包含一个指向其相关文档模板对象的指针
- **CFile**: 文件访问基类，实现文件输入与输出操作
- **CDC**: 设备描述表类，主要用来实现窗口绘制
 - **CPaintDC**: 绘图设备描述表
 - **CClientDC**: 客户区的设备描述表

1、MFC类的组织结构

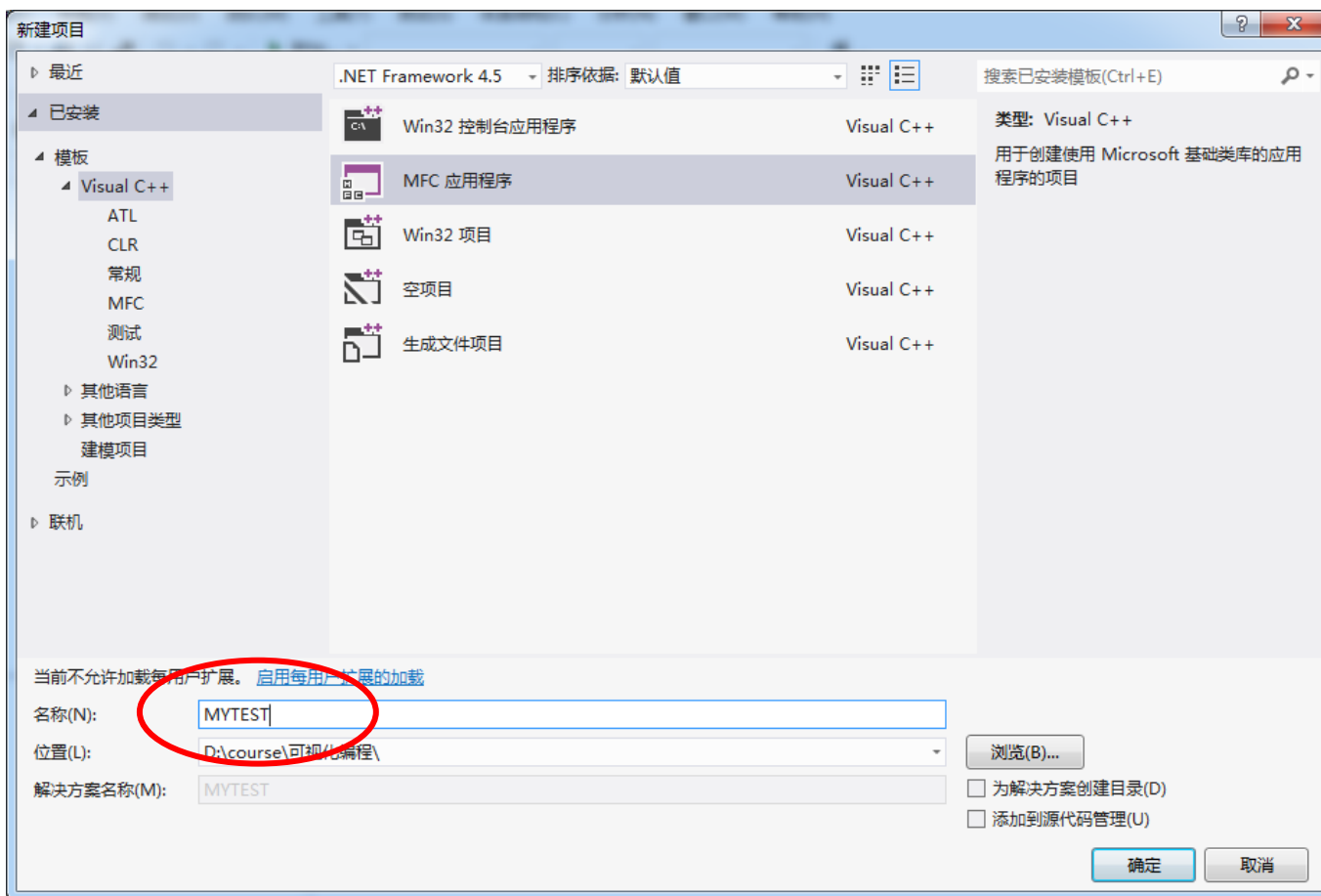
➤ MFC全局函数：以Afx为前缀

函数名	功能
AfxAbort	终止一个应用程序
AfxBeginThread	创建并执行一个线程
AfxEndThread	终止正在执行的线程
AfxMessageBox	弹出一个消息框
AfxGetApp	返回当前应用程序对象的指针
AfxGetInstanceHandle	返回当前应用程序对象的句柄
AfxRegisterWndClass	注册一个窗口类

➤ Afx 前缀函数代表应用程序框架(Application Framework)函数

2、MFC框架程序分析

➤ 新建一个名称为MYTEST的MFC应用程序



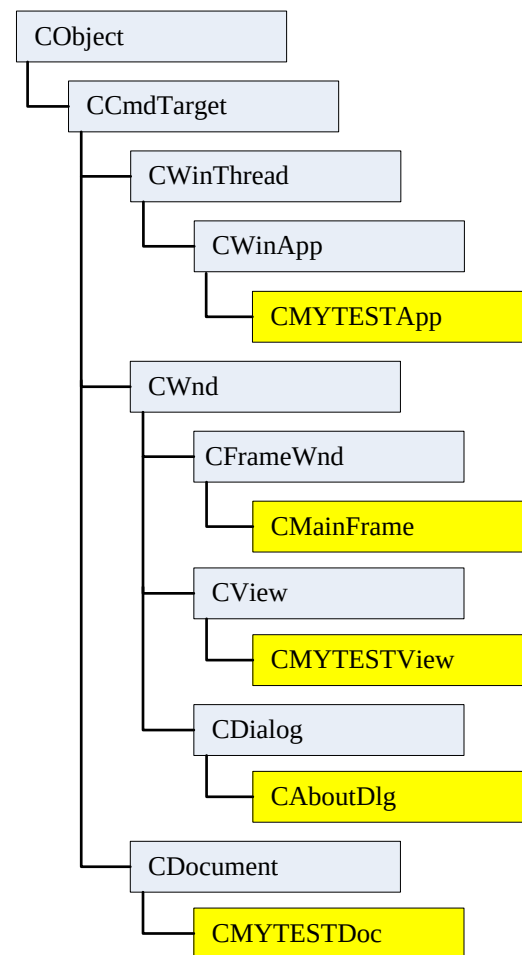
2、MFC框架程序分析

- 应用程序类型：单个文档，项目类型：MFC标准



2、MFC框架程序分析

➤ MFC AppWizard生成的五个类



复习：Windows应用程序的基本结构



➤ WinMain()——程序入口点

- 定义并注册窗口类：RegisterClassEx()
- 创建并显示窗口：CreateWindow()/ShowWindow()
- 消息循环：
GetMessage()/TranslateMessage()/DispatchMessage()

➤ WndProc()——窗口函数（回调函数）

- 使用switch-case区分消息
- 根据消息触发相应处理
- 调用DefWindowProc()处理应用程序不处理的消息及需要系统进一步处理的消息

2、MFC框架程序分析

1. 单步调试寻找WinMain()函数

系统程序

APPMODUL.CPP

```
extern "C" int WINAPI
_tWinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
_In_ LPTSTR lpCmdLine, int nCmdShow)
#pragma warning(suppress: 4985)
{
    // call shared/exported WinMain
    return AfxWinMain(hInstance, hPrevInstance, lpCmdLine,
nCmdShow);
}
```

- WinMain()函数在程序编译链接时，由链接器链接到MYTEST程序中的

2、MFC框架程序分析

2. MFC应用程序类

- CMYTESTApp类：CWinApp类的派生类
- theApp：CMYTESTApp类型的全局对象，唯一标识应用程序实例

MYTEST.h

```
class CMYTESTApp : public CWinApp
```

...

MYTEST.CPP

```
// 唯一的一个 CMYTESTApp 对象
```

```
CMYTESTApp theApp;
```

系统程序

- 设置断点调试程序，程序执行的顺序依次是：theApp全局对象定义处、CMYTESTApp类构造函数，WinMain()函数
- 调用CWinApp构造函数，实现MFC框架创建的类与MFC基类的关联

2、MFC框架程序分析

3. AfxWinMain()函数

系统程序

WINMAIN.CPP

```
int AFXAPI AfxWinMain(HINSTANCE hInstance, HINSTANCE
    hPrevInstance, _In_ LPTSTR lpCmdLine, int nCmdShow) {
    ...

    CWinThread* pThread = AfxGetThread();
    CWinApp* pApp = AfxGetApp();          //① 获得theApp全局对象
    ...

    if (pApp != NULL && !pApp->InitApplication())
    ...                                // ② 完成MFC内部管理
    if (!pThread->InitInstance())        // ③ 调用CMYTESTApp的
    ...                                // InitInstance()函数，完成窗口操作
    nReturnCode = pThread->Run();        // ④ 消息循环
    ... }
```

MFC框架窗口

➤ 窗口类的定义与注册

`AfxEndDeferRegisterClass()`: 首先判断窗口类的类型, 然后赋予其相应的类名, 之后调用`AfxRegisterClass()`函数注册窗口类

➤ 窗口的创建

`CWnd::CreateEx()`: 生成`CREATESTRUCT`结构 (用于设置窗口外观), 调用`CreateWindowEx()`函数创建窗口

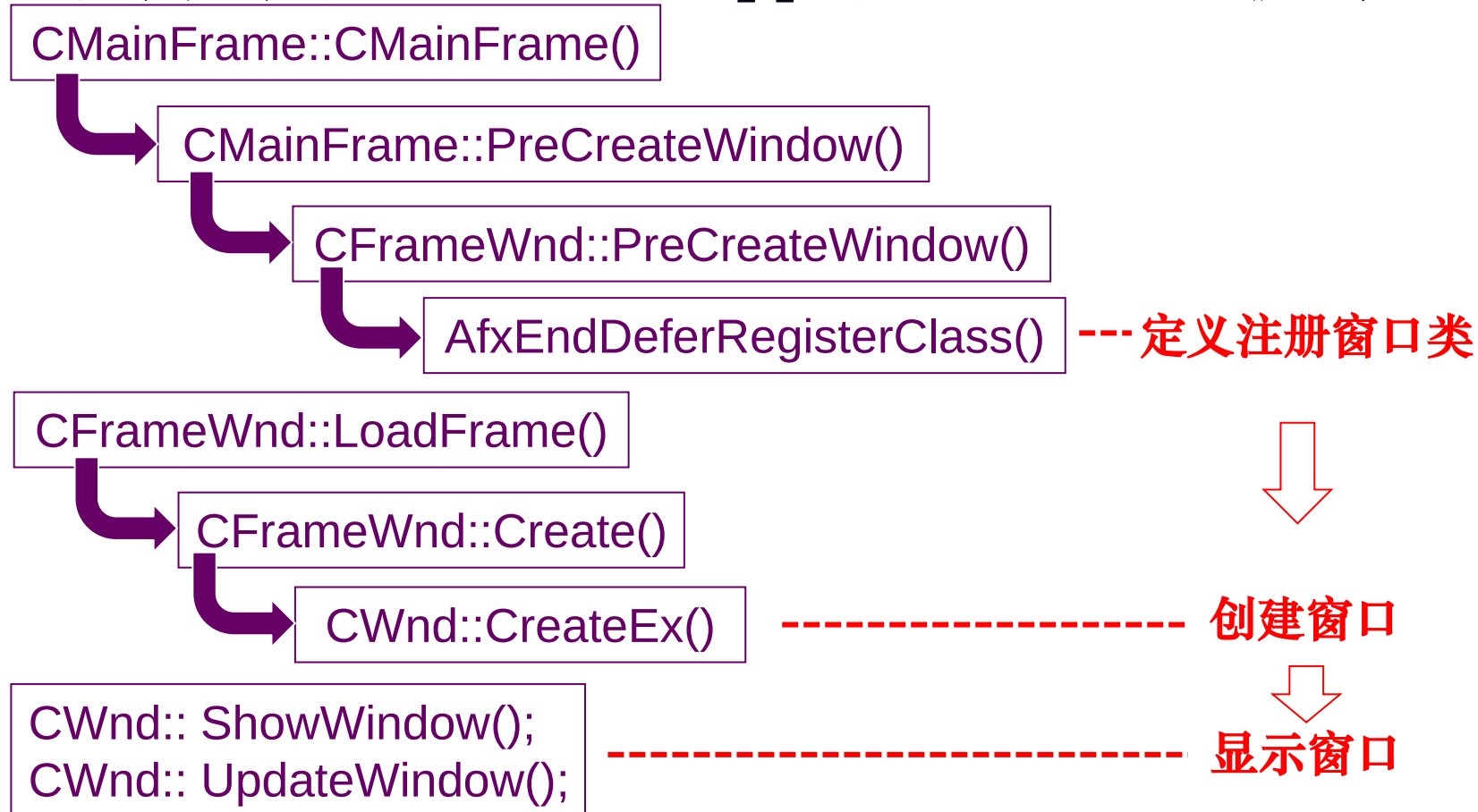
➤ 窗口的显示与更新(在生成文件中)

- `m_pMainWnd->ShowWindow(SW_SHOW);`
- `m_pMainWnd->UpdateWindow();`

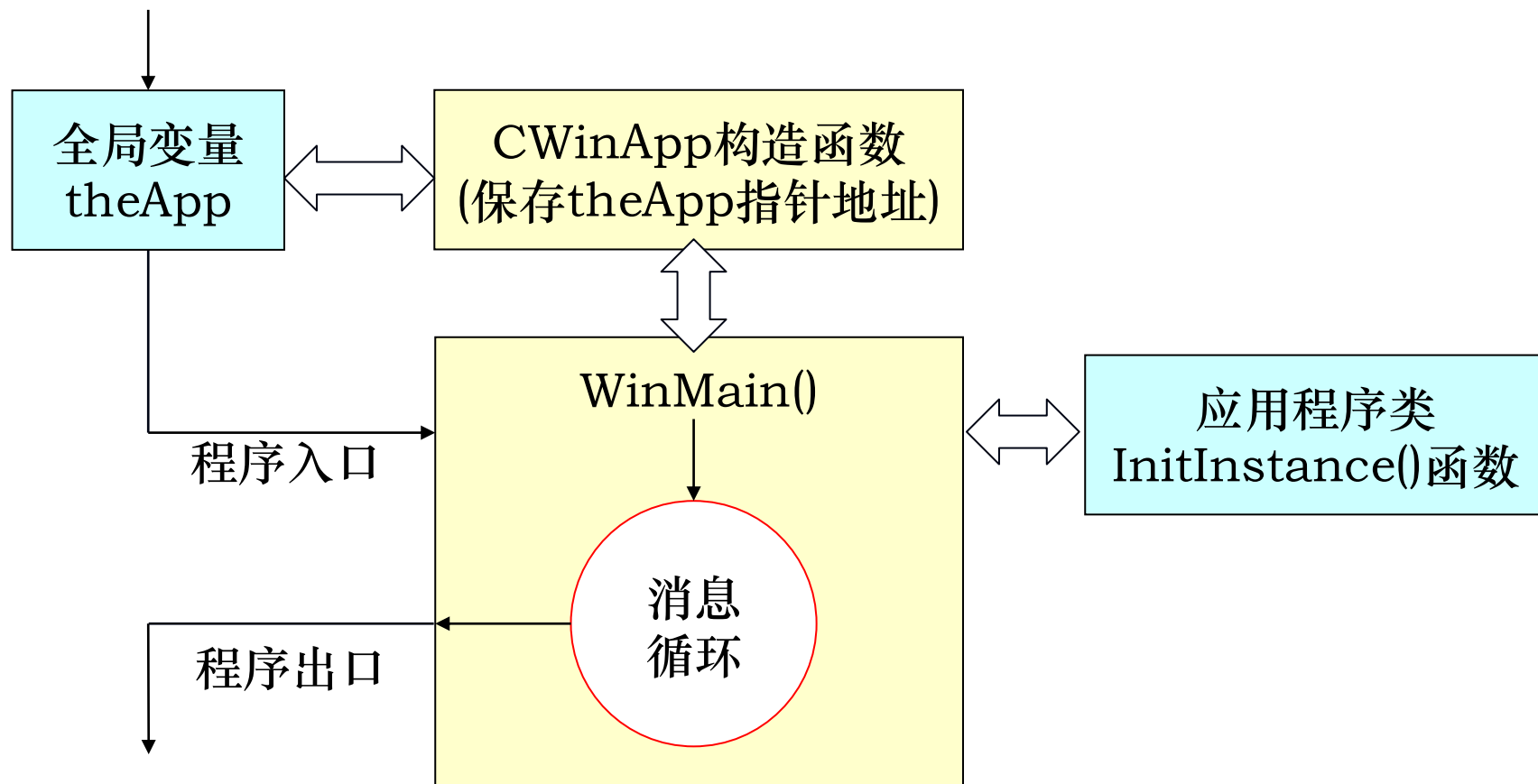
应用程序类`CMYTESTApp`的成员变量,
保存框架窗口类`CMainFrame`对象的指针

2、MFC框架程序分析

4. 应用程序类CMYTESTApp的InitInstance()函数



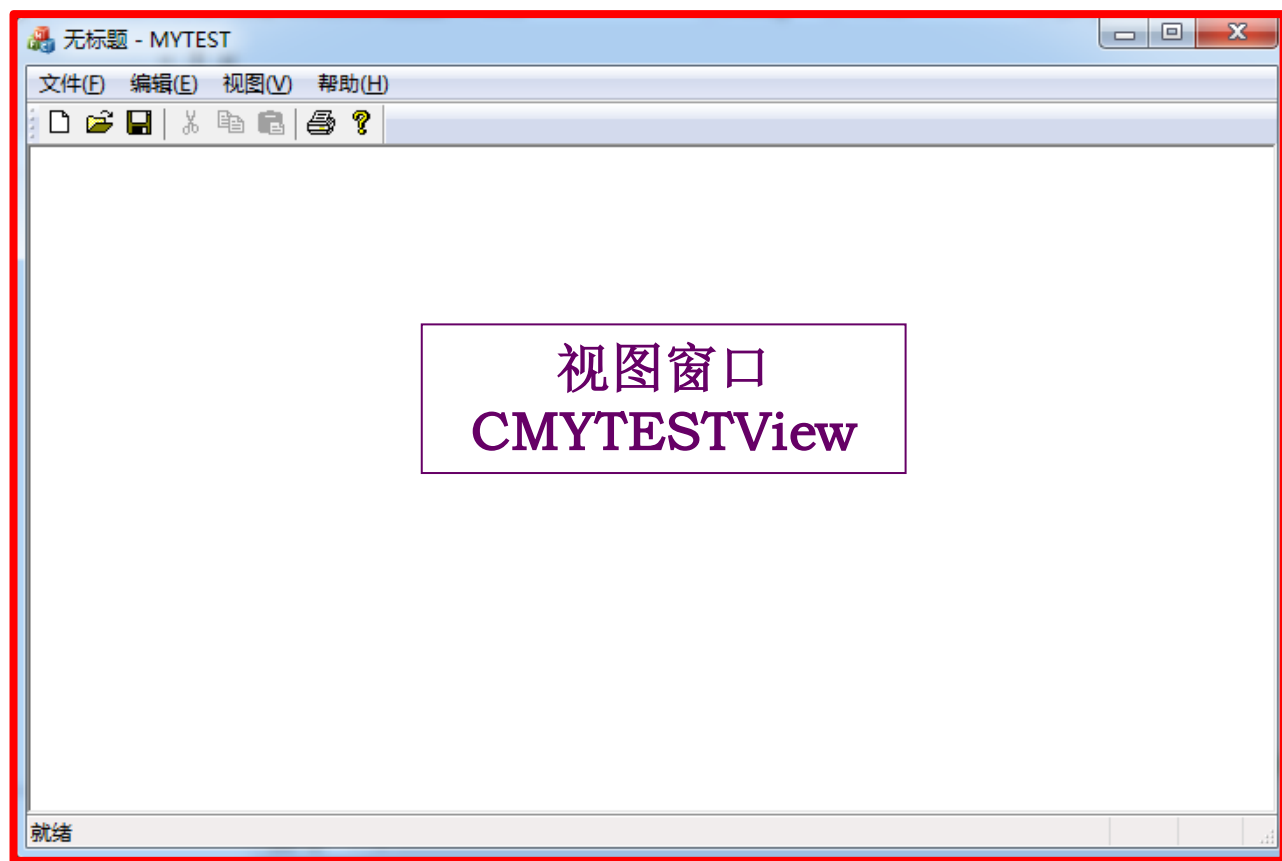
总结：MFC框架程序分析



总结：MFC框架程序分析

1. 利用全局应用程序对象 theApp 启动应用程序
2. 调用应用程序类的构造函数：首先调用其基类CWinApp的构造函数，后者完成应用程序的一些初始化工作，并将应用程序对象的指针保存起来
3. 进入 WinMain()函数：调用应用程序类的InitInstance()函数，完成应用程序的一些初始化工作，包括窗口类的定义、创建，窗口的显示和更新。期间会多次调用CreateEx()函数，因为一个单文档MFC 应用程序有多个窗口，包括框架窗口、工具条、状态条等
4. 进入消息循环：采用消息映射机制来处理各种消息的
5. 结束：当收到WM_QUIT 消息时，退出消息循环，程序结束

文档与视图



主框架窗口
CMainFrame

视图窗口
CMYTESTView

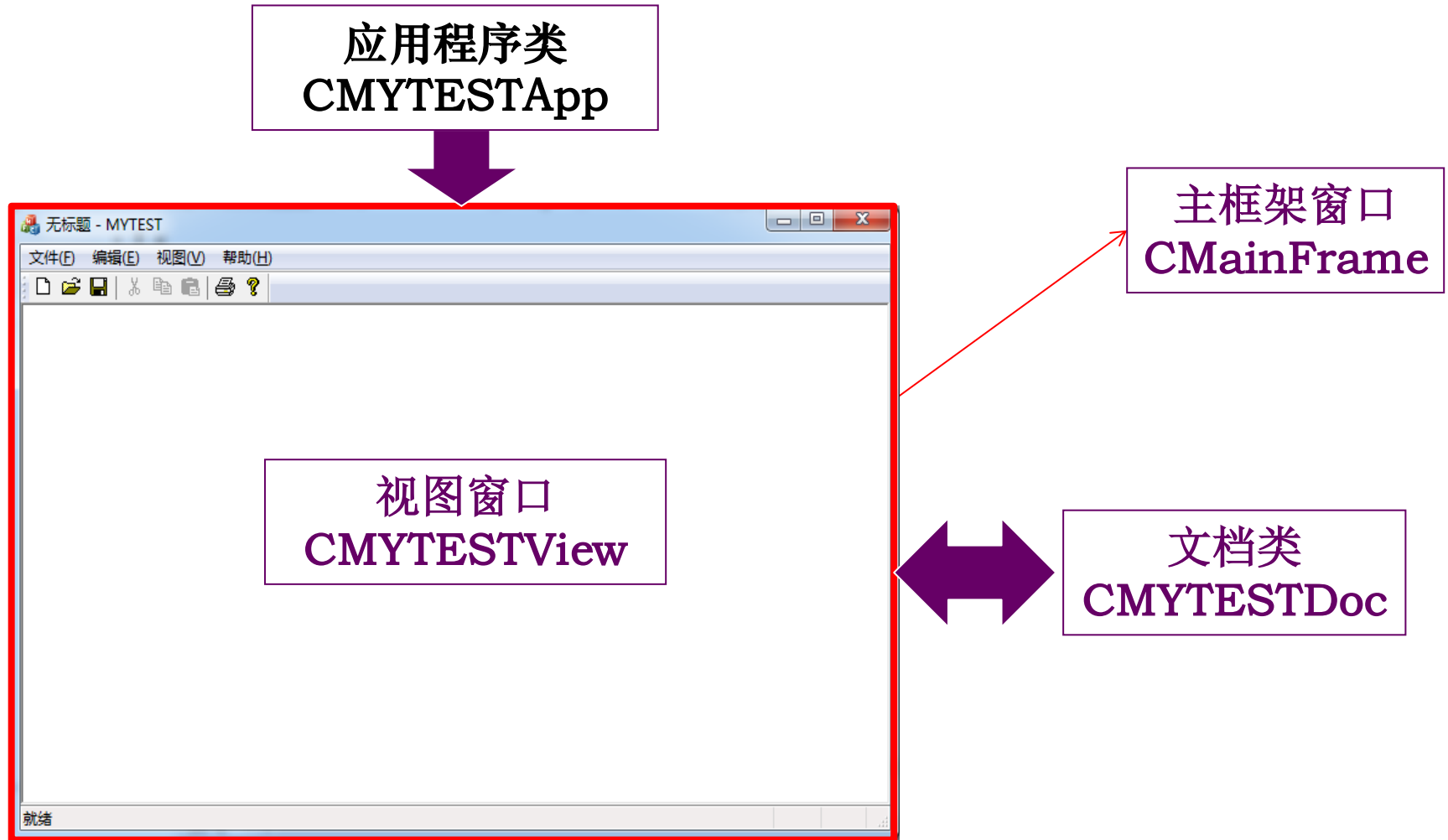
文档与视图

➤ 应用程序类CMYTESTApp的InitInstance()函数

系统程序

```
...  
// 注册应用程序的文档模板。文档模板  
// 将用作文档、框架窗口和视图之间的连接  
CSingleDocTemplate* pDocTemplate;    // 单文档模板对象指针  
pDocTemplate = new CSingleDocTemplate(  
    IDR_MAINFRAME,  
    RUNTIME_CLASS(CMYTESTDoc),        // 文档类  
    RUNTIME_CLASS(CMainFrame),        // 框架窗口  
    RUNTIME_CLASS(CMYTESTView));      // 视图类  
if (!pDocTemplate)  
    return FALSE;  
AddDocTemplate(pDocTemplate);          // 加载文档模板  
...
```

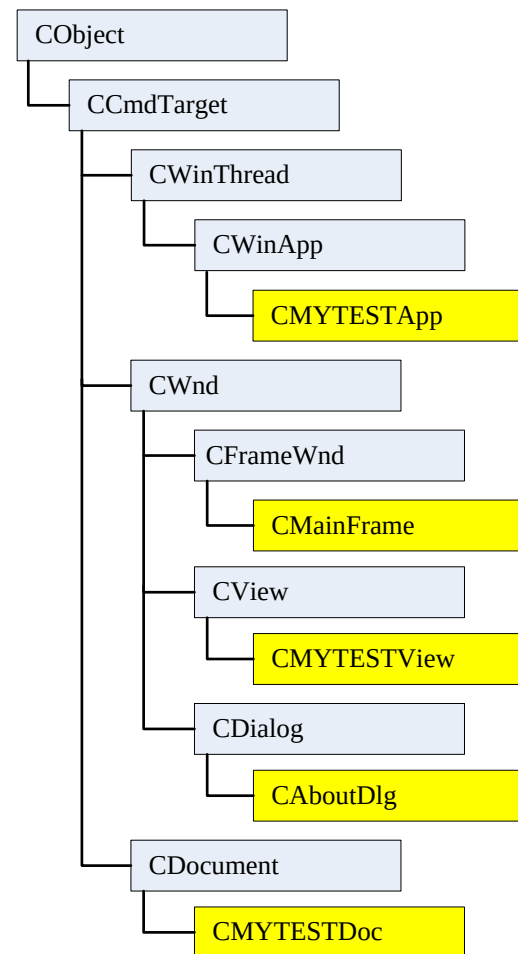
总结：MFC框架程序分析



总结：MFC框架程序分析

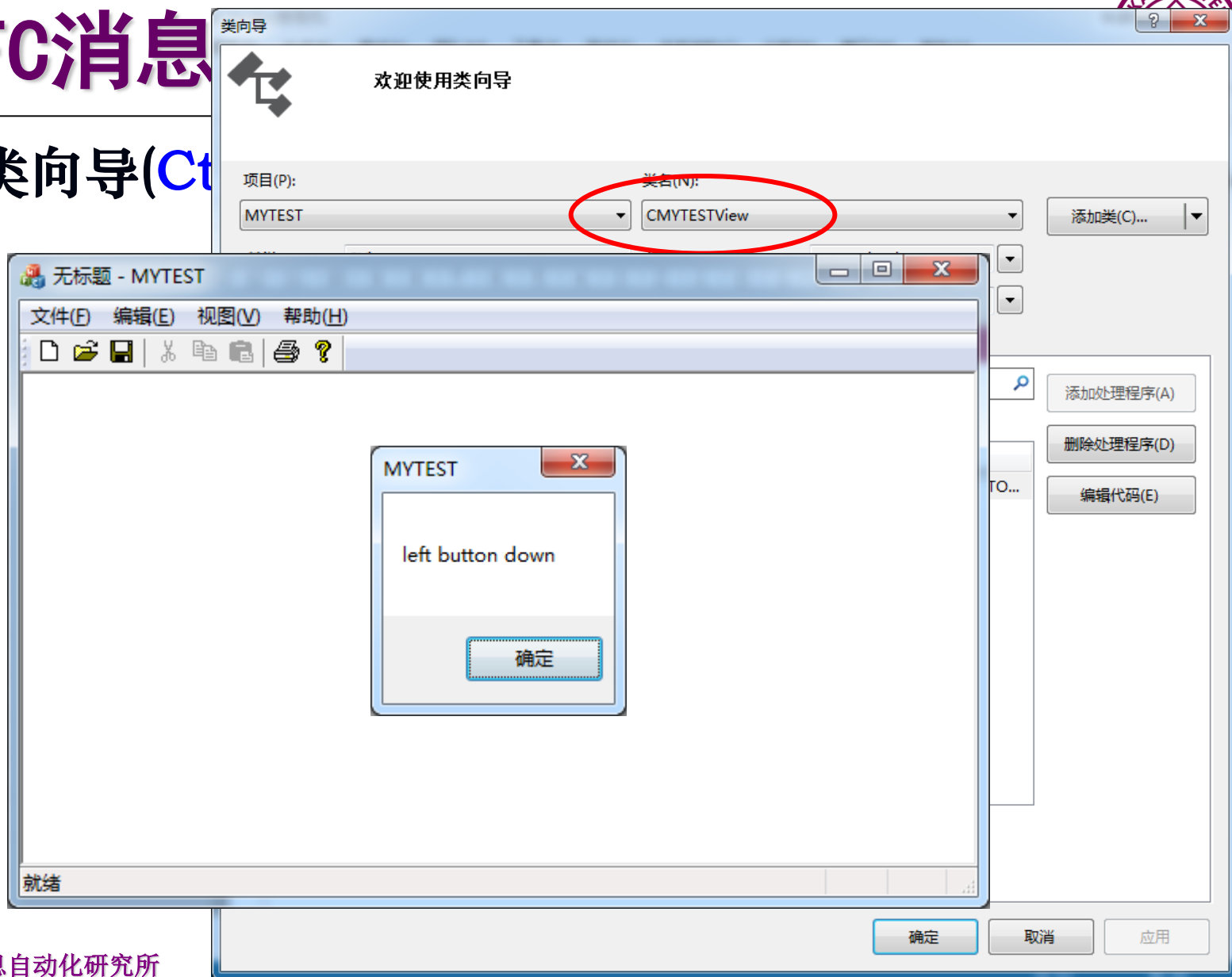
➤ MFC AppWizard生成的五个类

- CMYTESTApp：应用程序类，与WinMain()函数配合，管理应用程序初始化，负责保持文档、视图、框架窗口之间的关系，并启动消息循环
- CMainFrame：主框架窗口类，提供应用程序主窗口，包含最大/最小化按钮、标题栏、系统菜单、工具条和状态条
- CMYTESTDoc：文档类，对数据进行管理和维护
- CMYTESTView：视图窗口类，是文档和用户的中介，在屏幕上显示文档数据，视图窗口覆盖在主框架窗口之上
- CAboutDlg：对话框类



3、MFC消息

➤ 通过类向导(Class Wizard)



3、MFC消息映射机制

➤ 加入消息响应函数后，源文件增加的代码

1. 消息响应函数原型

系统添加

MYTESTView.h

protected:

DECLARE_MESSAGE_MAP() // 生成的消息映射函数

public: // 消息响应函数的声明

afx_msg void OnLButtonDown(UINT nFlags, CPoint point);

3、MFC消息映射机制

➤ 加入消息响应函数后，源文件增加的代码

1. 消息响应函数原型
2. 消息响应函数

MYTESTView.cpp

```
void CMYTESTView::OnLButtonDown(UINT nFlags, CPoint  
    point)  
{  
    // TODO: 在此添加消息处理程序代码和/或调用默认值  
    MessageBox(_T("left button down"));  
    CView::OnLButtonDown(nFlags, point);  
}
```

添加蓝色
代码



3、MFC消息映射机制

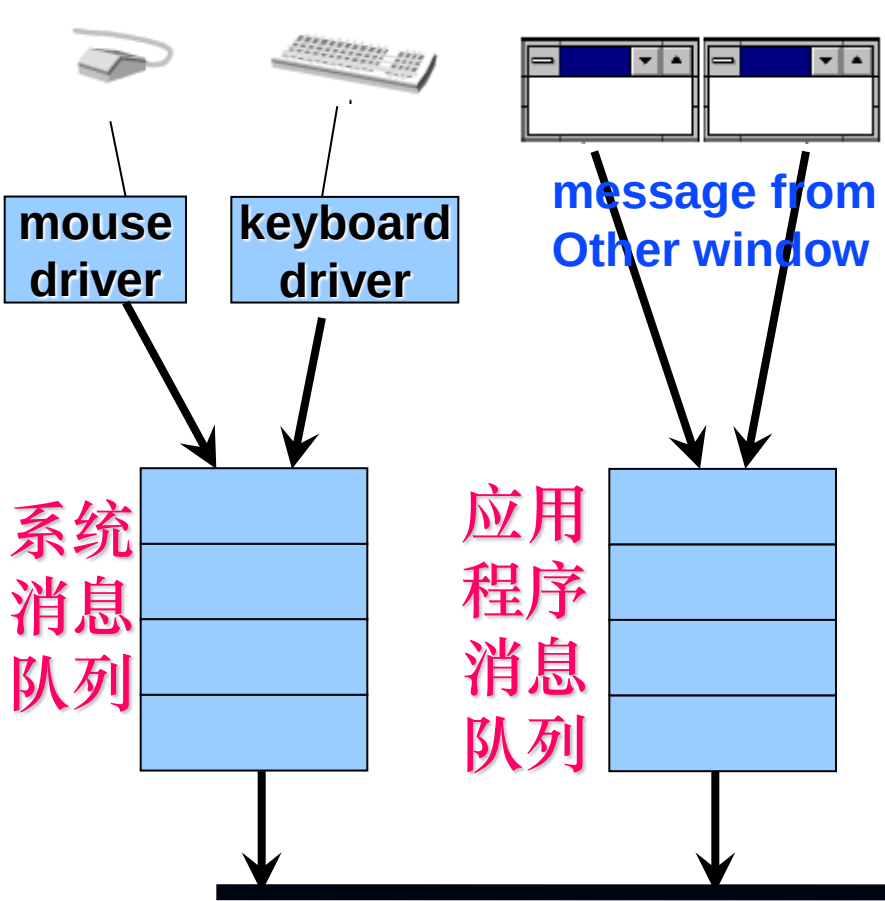
- 加入消息响应函数后，源文件增加的代码
 1. 消息响应函数原型
 2. 消息响应函数
 3. 消息映射宏：将消息和消息响应函数关联起来

系统添加

MYTESTView.cpp

```
BEGIN_MESSAGE_MAP(CMYTESTView, CView)
...
    ON_WM_LBUTTONDOWN()
END_MESSAGE_MAP()
```

```
#define ON_WM_LBUTTONDOWN() \
{ WM_LBUTTONDOWN, 0, 0, 0, AfxSig_vwp, \
  (AFX_PMSG)(AFX_PMSGW) \
  (static_cast< void (AFX_MSG_CALL CWnd::*)(UINT, CPoint) > ( &ThisClass :: OnLButtonDown)) },
```



WinMain ()

```
{
  初始化;
  while(GetMessage(&msg)){
    TranslateMessage( );
    DispatchMessage( );
  }
  system
  WndProc(msg)
  {
    switch(msg){
    case MSG1:
    case MSG2:
    }
    return 0;
  }
}
```

复习:
Windows消息处理机制

3、MFC消息映射机制

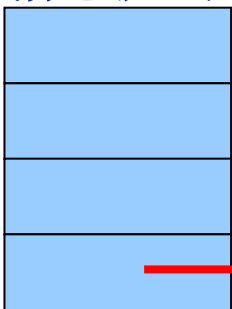
➤ 消息映射表

- 在每个能接收和处理消息类中，定义一个消息映射表，给出消息和消息函数的静态对照
- 在消息映射表中，消息与对应的消息处理函数指针是成对出现的
- 某个类能处理的所有消息及其对应的消息处理函数的地址都列在这个类对应的静态表中

WM_CREATE	OnCreate()
WM_LBUTTONDOWN	OnLButtonDown()
...	...

3、MFC消息映射机制

消息队列



MESSAGE_MAP消息映射表

WM_CREATE	OnCreate()
WM_LBUTTONDOWN	OnLButtonDown()
...	...

AfxWinMain()

```
{  
    pApp->InitApplication()  
    pThread->InitInstance()  
    nReturnCode = pThread->Run()  
}
```

CWinThread::Run()

CWinThread::PumpMessage ()

```
do {  
    ::SendMessage()  
    ::TranslateMessage()  
    ::DispatchMessage()  
} while()
```

举例

➤ 窗口消息

- WM_CREATE消息：当窗口对象生成后，系统发送WM_CREATE，完成窗口的初始化
- WM_DESTROY消息：当应用程序退出时，系统发送WM_DESTROY，完成某些收尾工作
- WM_PAINT消息：当窗口发生变化时，系统发送WM_PAINT，重绘窗口并显示数据

举例：窗口消息

- 添加WM_CREATE消息，编写WM_CREATE的响应函数

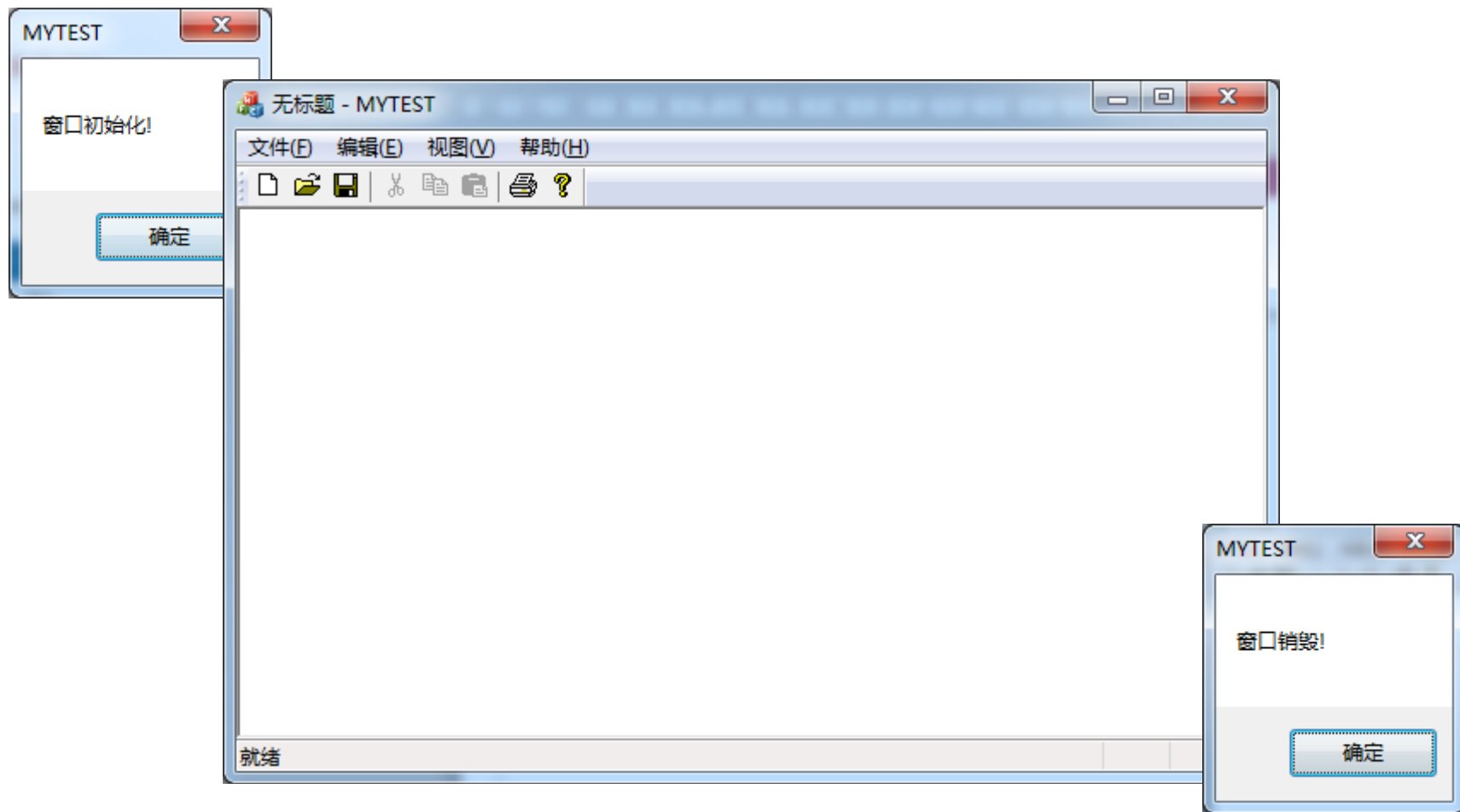
```
int CMYTESTView::OnCreate(LPCREATESTRUCT lpCreateStruct) {  
    if (CView::OnCreate(lpCreateStruct) == -1)    return -1;  
    MessageBox(_T("窗口初始化! "));  
    return 0;  
}
```

- 添加WM_DESTROY消息，编写响应函数

```
void CMYTESTView::OnDestroy() {  
    CView::OnDestroy();  
    // TODO: 在此处添加消息处理程序代码  
    MessageBox(_T("窗口销毁! "));  
}
```

添加蓝色
代码

举例：窗口消息



举例：窗口消息

- 在CMYTESTView类定义中

```
private:      int m_Num;
```

- 在CMYTESTView构造函数中

```
m_Num = 0;
```

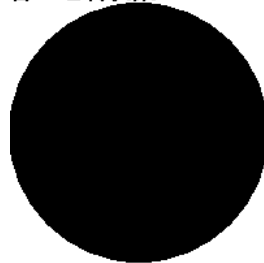
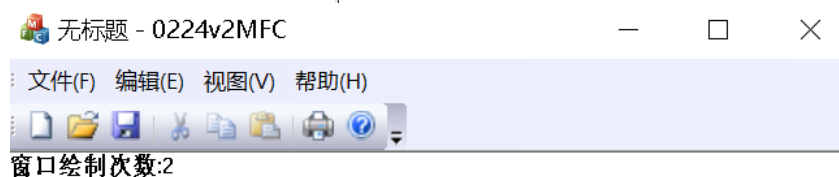
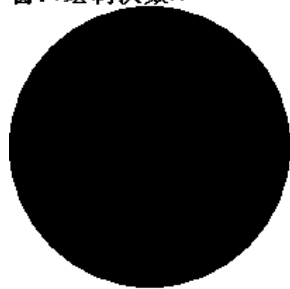
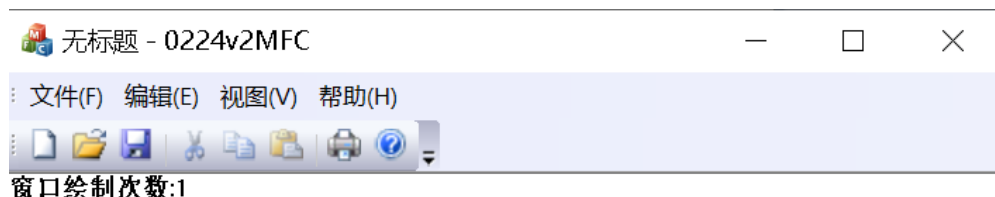
- 在CMYTESTView::OnDraw()中

```
CString str;  
str.Format(_T(“窗口绘制次数:%d”),m_Num++);
```

```
pDC->TextOut(0,0,str);  
pDC->SelectStockObject(BLACK_BRUSH);  
pDC->Ellipse(0,20,200,220);
```

****注意** *void*
CMYTESTView::OnDraw
(*CDC* pDC*) 把pDC默
认的注释去掉

➤ 通过改变窗口的大小，可以实现窗口的重新绘制



总结

- 初识可视化编程
- Windows应用程序
 1. Windows程序开发流程
 2. 事件与消息
 3. Windows应用程序的基本结构
 4. 应用程序举例
- MFC基础知识
 1. MFC类的组织结构
 2. MFC框架程序分析
 3. MFC消息映射机制

课后练习

- 阅读课本第一、二章
- 复习C++基础知识，包括指针、继承、多态性和虚函数